
Asymptotic Behavior

Big O, big Omega, big Theta — and the two witnesses that prove each one.

Today

AGENDA

- 01 Floor, ceiling, sigma, and pi
- 02 Big O, and the two witnesses that prove it
- 03 The growth hierarchy, with real numbers attached
- 04 Big Omega and big Theta
- 05 Theorems that let you skip the witnesses

when you reduce the time complexity
of your algorithm from $O(n)$ to $O(1)$



Floor and Ceiling

FLOOR

$$\lfloor x \rfloor$$

The **largest** integer that is less than or equal to x .

CEILING

$$\lceil x \rceil$$

The **smallest** integer that is greater than or equal to x .

$$\lfloor \tfrac{1}{2} \rfloor = 0$$

$$\lceil \tfrac{1}{2} \rceil = 1$$

$$\lfloor -3.1 \rfloor = -4$$

$$\lceil -3.1 \rceil = -3$$

The negatives are the trap. $\lfloor -3.1 \rfloor$ is -4 , because -4 is the largest integer that does not exceed -3.1 . Floor is not “chop off the decimals.”

Why We Need This

n	binary search $\lceil \log_2 n \rceil$	linear search n	bubble sort $n(n-1)/2$
6	3	6	15
100	7	100	4,950
1,000	10	1,000	499,500
1,000,000	20	10^6	$\approx 5 \times 10^{11}$

These are the counts we derived by hand last lecture.

At $n = 6$ they look comparable. At a million they are not in the same universe.

We want to say “that last column grows like n^2 ” and never write $n(n-1)/2$ again. The constant $\frac{1}{2}$ and the $-n/2$ are noise.

Big O

BIG O

DEFINITION

Let f and g be functions to the real numbers. We say $f(n)$ is $O(g(n))$ if there are positive constants C and k such that

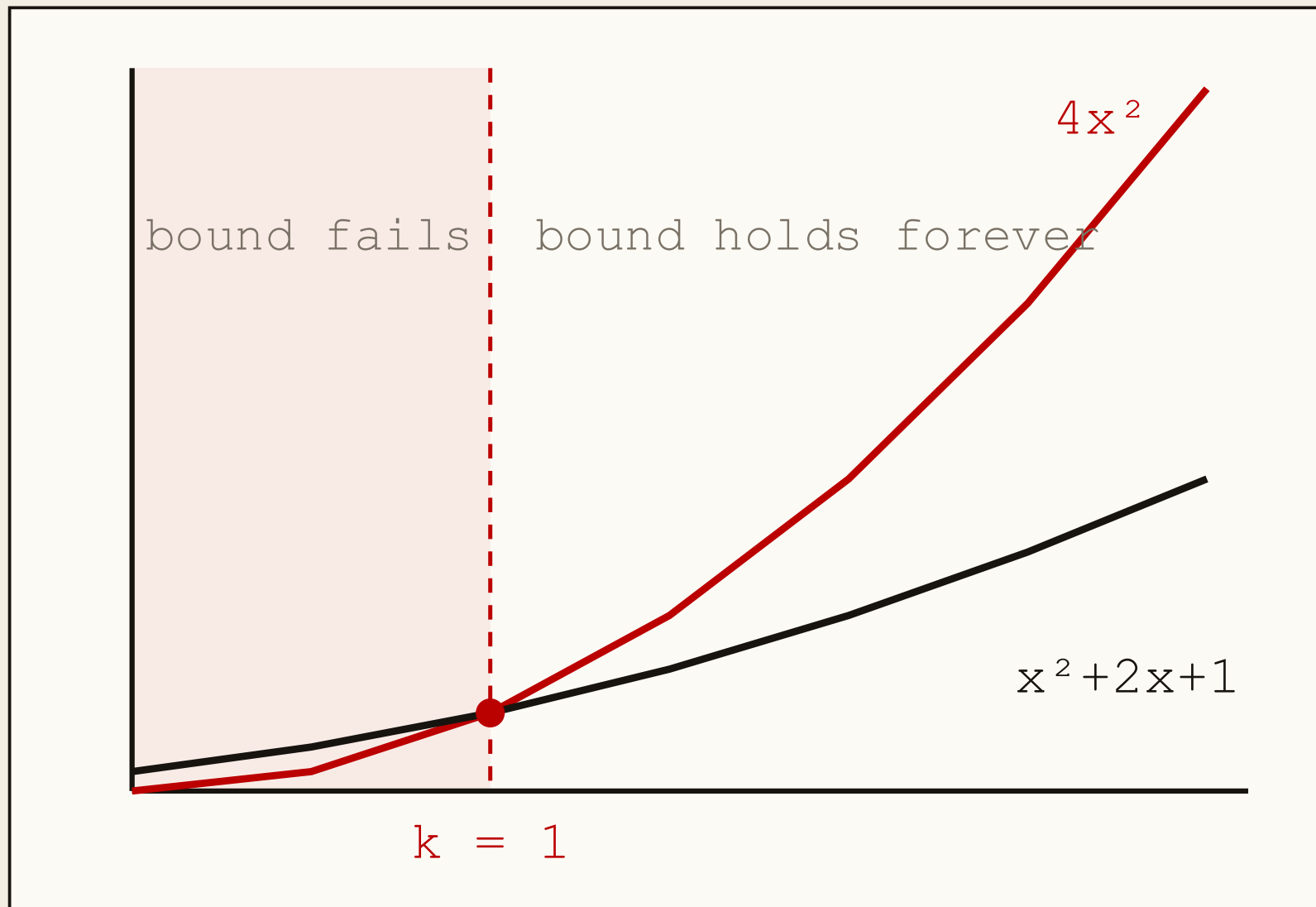
$$|f(n)| \leq C |g(n)| \text{ whenever } n > k$$

C and k are called **witnesses**. Producing a pair of them is the proof.

Also written $f(n) \in O(g(n))$ or $f(n) \preccurlyeq g(n)$. The quantifier shape is *there exist* C, k such that *for all* $n > k$ — straight from Lecture 2.

What the Witnesses Mean

BIG O



Left of k the bound really does fail — the black curve sits above the scarlet one.

Right of k it holds, and keeps holding.

Big O makes no claim about small inputs. That is precisely what k is for — it lets you ignore the beginning of the graph.

Example: $x^2 + 2x + 1$ is $O(x^2)$

BIG O

We need C and k with $x^2 + 2x + 1 \leq Cx^2$ for all $x > k$.

WHEN $x > 1$

$$x < x^2 \text{ and } 1 < x^2$$

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

$$C = 4, k = 1$$

Bound every lower-order term by the leading one, then add the coefficients: $1 + 2 + 1 = 4$. The condition $x > 1$ is what licenses each replacement.

Example: n^2 Is Not $O(n)$

BIG O

To show n^2 is not $O(n)$ we must show that *no* pair of witnesses exists. That is a negative claim, so we use **contradiction**.

SUPPOSE THEY DO

$$n^2 \leq Cn \text{ whenever } n > k$$

divide by $n > 0$

$$n \leq C \text{ whenever } n > k$$

No matter what C and k are, $n \leq C$ cannot hold for *all* $n > k$ — take $n = \max(k, C) + 1$ and it fails.

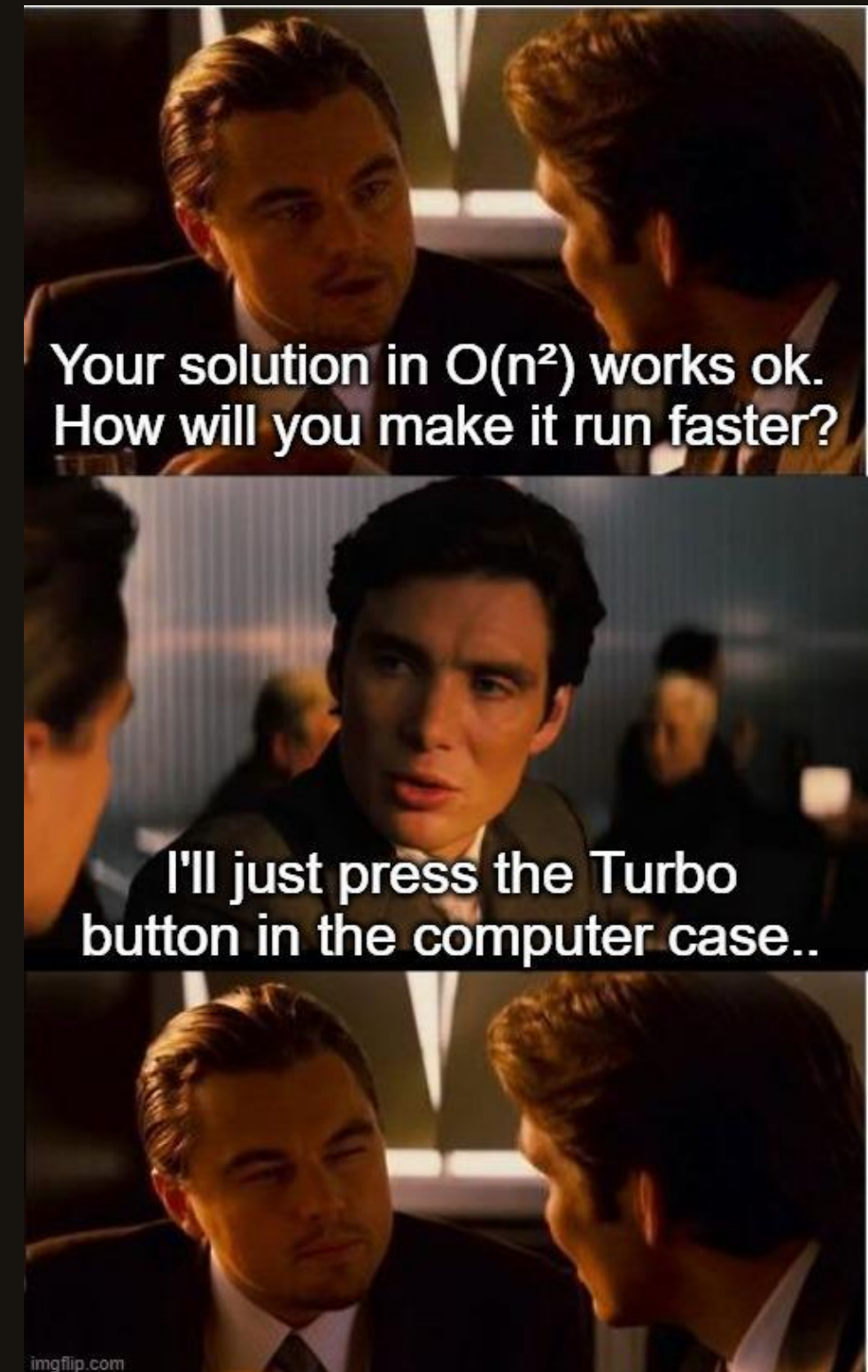
Contradiction. ■

The shape to remember: proving *is* big O means exhibiting witnesses; proving *is not* big O means refuting every possible pair.

$C = 4$ OR $C = 4,000,000$

Big O does not care about your constant.

Which is mathematically clean and occasionally infuriating.
A hundredfold slowdown is invisible to O and extremely visible to your users.



Sigma and Pi Notation

TOOLKIT

SUM

$$\sum_{i=1}^n a_i = a_1 + a_2 + \cdots + a_n$$

$$\sum_{i=1}^n i = n(n+1)/2$$

$$\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$$

PRODUCT

$$\prod_{i=1}^n a_i = a_1 a_2 \cdots a_n$$

$$\prod_{i=1}^n i = n!$$

$$2! = 2, 3! = 6, 4! = 24$$

That first closed form — $n(n+1)/2$ — is bubble sort's comparison count with the index shifted by one. The formulas are not abstract; you counted one of them by hand last lecture.

Polynomials, and a Sum

BIG O

THEOREM

If $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ with real coefficients, then $f(x) \in O(x^n)$.

Only the highest power matters. Everything else is absorbed into C .

EXAMPLE — SHOW $1 + 2 + \dots + n \in O(n^2)$

$$1 + 2 + \dots + n \leq n + n + \dots + n = n \cdot n = n^2$$

Each of the n terms is at most n . Witnesses $C = 1, k = 1$.

Example: $n!$ and $\log n!$

BIG O

REPLACE EVERY FACTOR BY n

$$n! = (1)(2) \cdots (n) \leq (n)(n) \cdots (n) = n^n$$

So $n! \in O(n^n)$, with $C = k = 1$.

NOW TAKE LOGS OF THE SAME INEQUALITY

$$\log(n!) \leq \log(n^n) = n \log n$$

So $\log n! \in O(n \log n)$, again with $C = k = 1$.

Taking logarithms turns a product into a sum. That move appears constantly, and it is why $n \log n$ shows up in sorting bounds.

Example: $n \in O(2^n)$ and $\log n \in O(n)$

BIG O

Take as given that $n < 2^n$ for every positive integer n — we prove it by induction later in the course.

DIRECTLY

$$n < 2^n$$

Witnesses $C = k = 1$ give $n \in O(2^n)$.

TAKE $\log_2$ OF BOTH SIDES

$$\log n < \log 2^n = n \log 2 = n$$

Same witnesses give $\log n \in O(n)$.

Base 2 throughout, so $\log 2 = 1$. Changing base only multiplies by a constant, and big O absorbs constants — which is why nobody writes the base.

The Growth Hierarchy

HIERARCHY

$$1 \ll \log n \ll n \ll n \log n \ll n^2 \ll 2^n \ll n!$$

Here $\ll$ means big O: $\log n \ll n$ is exactly $\log n \in O(n)$.

Every $\ll$ here is strict: the reverse is false in every case.

The Same Hierarchy, in Numbers

HIERARCHY

n	log ₂ n	n	n log ₂ n	n ²	2 ⁿ	n!
10	3.3	10	33	100	1,024	3,628,800
100	6.6	100	664	10,000	≈ 10 ³⁰	≈ 10 ¹⁵⁸
1,000	10	1,000	9,966	1,000,000	≈ 10 ³⁰¹	≈ 10 ²⁵⁶⁸

A machine doing a billion operations per second clears the n^2 column in a millisecond and never finishes the last two.

For scale: the observable universe holds roughly 10^{80} atoms. Both scarlet entries in the middle row already exceed it.

Four Facts That Save You Work

HIERARCHY

01 $d > c > 1 \Rightarrow n^c \in O(n^d)$

Higher power wins. $n^3 \in O(n^{10})$, and $n^{10} \notin O(n^3)$.

02 $c, d > 0 \Rightarrow (\log n)^c \in O(n^d)$

Any polynomial beats any power of a log. $(\log n)^{100} \in O(n^{0.01})$.

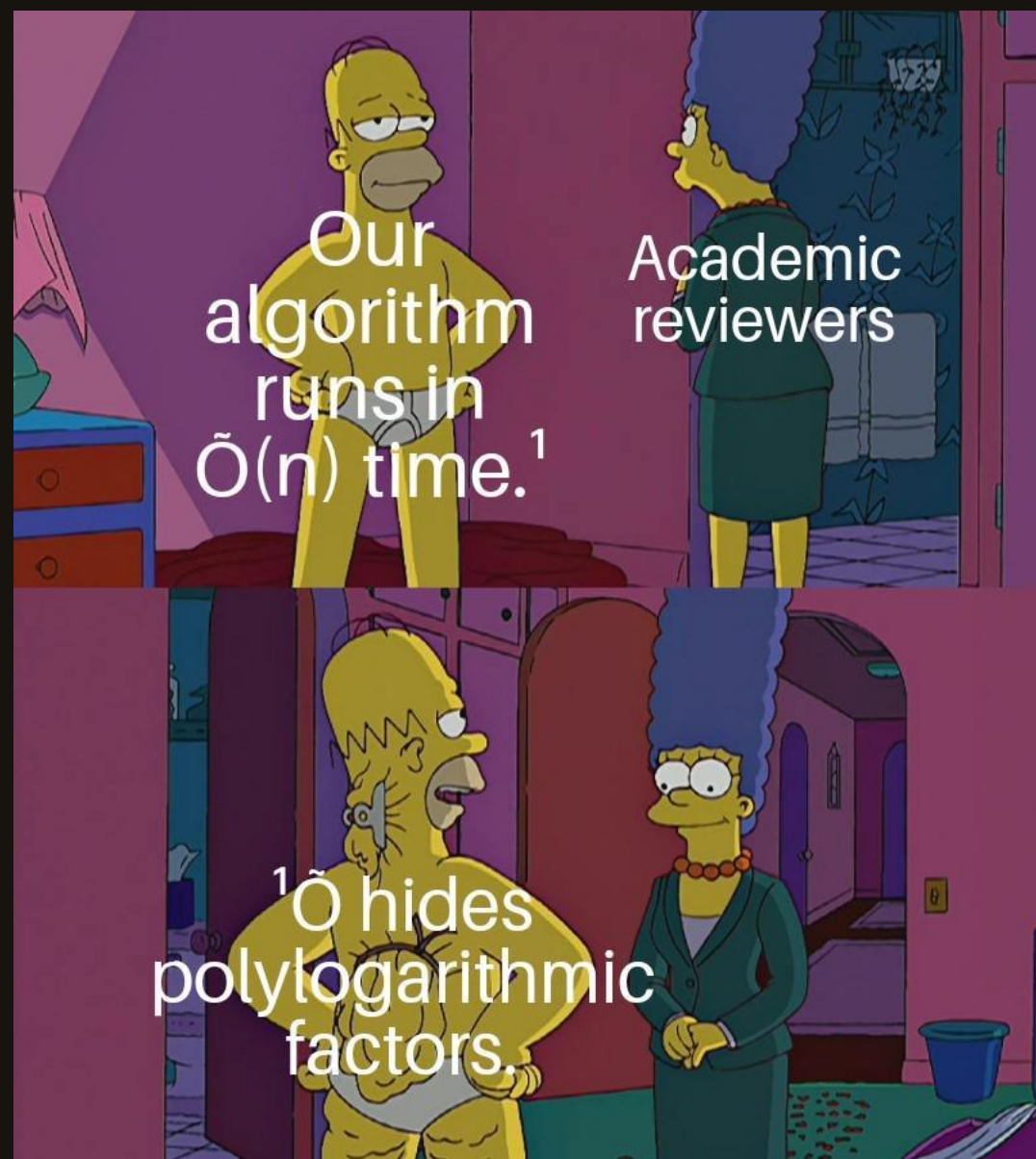
03 $d > 0, b > 1 \Rightarrow n^d \in O(b^n)$

Any exponential beats any polynomial. $n^{100} \in O(2^n)$.

04 $c > b > 1 \Rightarrow b^n \in O(c^n)$

Bigger base wins. $3^n \in O(10^n)$, and $10^n \notin O(3^n)$.

In every case the reverse containment **fails**. Rule 2 is the one that surprises people: $(\log n)^{100}$ really does lose to $n^{0.01}$, eventually.



FOR SUFFICIENTLY LARGE n

“Eventually” is doing a lot of work in that sentence.

$(\log n)^{1.00} \in O(n^{0.01})$ is true, and the crossover happens at an n you will never run. Both facts matter.

Big Omega

Ω AND Θ

DEFINITION

$f(x)$ is $\Omega(g(x))$ if there are positive constants C and k such that $|f(x)| \geq C|g(x)|$ whenever $x > k$.

$$f(x) \in \Omega(g(x)) \iff g(x) \in O(f(x))$$

Same inequality, read from the other side. Big O is a ceiling; Ω is a floor.

You never need a new proof technique for Ω — swap the two functions and prove a big- O statement instead.

Big Theta

Ω AND Θ

DEFINITION

$f(x)$ is $\Theta(g(x))$ if there are positive constants C_1, C_2 and k such that

$$C_2 |g(x)| \leq |f(x)| \leq C_1 |g(x)| \text{ whenever } x > k$$

$$f \in \Theta(g) \Leftrightarrow f \in O(g) \text{ and } f \in \Omega(g)$$

Squeezed from both sides, so f and g grow at the same rate up to constants. Also written $f(x) \approx g(x)$. When someone asks how fast an algorithm is, Θ is the honest answer.

Example: $\sqrt[3]{(3n^3 + 2n^2 - n + 1)} \in \Theta(n)$

Ω AND Θ

We need C_1, C_2, k with $C_2n \leq \sqrt[3]{(3n^3 + 2n^2 - n + 1)} \leq C_1n$ for all $n > k$. Assume $n > 1$ throughout.

UPPER BOUND

drop $-n$, then push $2n^2$ and 1 up to n^3

$$\sqrt[3]{(3n^3 + 2n^2 - n + 1)}$$

$$< \sqrt[3]{(3n^3 + 2n^3 + n^3)}$$

$$= \sqrt[3]{(6n^3)} = \sqrt[3]{6} \cdot n$$

LOWER BOUND

drop $2n^2$ and $+1$, then use $n^3 > n$

$$\sqrt[3]{(3n^3 + 2n^2 - n + 1)}$$

$$> \sqrt[3]{(3n^3 - n)}$$

$$> \sqrt[3]{(3n^3 - n^3)} = \sqrt[3]{2} \cdot n$$

$$C_1 = \sqrt[3]{6}, \quad C_2 = \sqrt[3]{2}, \quad k = 1$$

Upper bounds get generous, lower bounds get stingy. Both directions only ever push terms toward n^3 , because that is what the cube root cancels.

When Θ Fails

To break Θ you only need *one* of the two containments to fail.

$$n^3 \in O(n^{10}) \quad \checkmark$$

$$n^{10} \notin O(n^3) \quad \times$$

$$\text{so } n^3 \notin \Omega(n^{10})$$

$$\text{so } n^3 \notin \Theta(n^{10})$$

$$n \log n \in O(n^2) \quad \checkmark$$

$$n^2 \notin O(n \log n) \quad \times$$

$$\text{so } n \log n \notin \Omega(n^2)$$

$$\text{so } n \log n \notin \Theta(n^2)$$

Both fail on the Ω side, which is the usual situation: the big-O bound is true but loose. “ $n \log n$ is $O(n^2)$ ” is correct and unhelpful.

Two Theorems

THEOREMS

POLYNOMIALS, UPGRADED TO Θ

If $f(x) = a_n x^n + \dots + a_1 x + a_0$ with $a_n \neq 0$, then $f(x) \in \Theta(x^n)$.

SUMS WITH A VARIABLE LIMIT

$$\sum_{i=1}^{f(n)} i^k \in \Theta(f(n)^{k+1})$$

Example: $\sum_{i=1}^{\log n} i^{1^0} \in \Theta((\log n)^{1+1})$

Sanity check on the second one with $f(n) = n$ and $k = 1$: it says $1 + 2 + \dots + n \in \Theta(n^2)$, which we prove by hand in two slides.

Combining Θ Estimates

THEOREMS

Suppose $f_1 \in \Theta(g_1)$ and $f_2 \in \Theta(g_2)$. Then:

$(f_1 + f_2)(n) \in \Theta(\max\{g_1, g_2\})$ The bigger term swallows the smaller one.

$(f_1 \cdot f_2)(n) \in \Theta(g_1 \cdot g_2)$ Products just multiply.

The sum rule is why nested loops with different bounds are easy and why a single dominant term decides everything.

Example: $(x + 1)\log(x^2 + 1) + 3x^2$

THEOREMS

Take each summand separately, then apply the sum rule.

$$(x + 1) \log(x^2 + 1) \approx x \log x^2 = 2x \log x \approx x \log x$$

$$3x^2 \approx x^2$$

$$f(x) \in \Theta(\max\{x \log x, x^2\}) = \Theta(x^2)$$

Two constants got dropped along the way — the 2 from $\log x^2 = 2 \log x$ and the 3 from $3x^2$. Θ absorbs both. And $x \log x \leq x^2$, so the max is x^2 .

Example: $1 + 2 + \dots + n \in \Theta(n^2)$

THEOREMS

The upper bound is done: $1 + 2 + \dots + n \leq n^2$. We still need the lower bound, $\Omega(n^2)$.

THROW AWAY THE SMALL HALF

$$\begin{aligned}\sum_{i=1}^n i &\geq \sum_{i=n/2}^n i = n/2 + (n/2 + 1) + \dots + n \\ &> n/2 + n/2 + \dots + n/2 = (n/2)(n/2) = n^2/4\end{aligned}$$

$$n^2/4 < 1 + 2 + \dots + n \leq n^2$$

$$C_1 = 1, C_2 = 1/4, k = 1$$

Keep the constants straight: C_1 is the *upper* multiplier and C_2 the *lower* one, matching $C_2|g| \leq |f| \leq C_1|g|$. Swapping them is the most common slip on this problem.

Floor and Ceiling Are $\Theta(x)$

THEOREMS

For $x > 2$ we have $x/2 > 1$, hence $x - 1 > x/2$.

TWO FACTS ABOUT THE FLOOR

$$\lfloor x \rfloor \leq x$$

$$\lfloor x \rfloor > x - 1 > x/2$$

$$x/2 < \lfloor x \rfloor \leq x \Rightarrow \lfloor x \rfloor \in \Theta(x)$$

For the ceiling: when x is not an integer, $\lceil x \rceil = \lfloor x \rfloor + 1 \approx \lfloor x \rfloor$, so $\lceil x \rceil \in \Theta(x)$ too.

This is the licence for writing binary search's cost as $\Theta(\log n)$ and never mentioning the ceiling again.

Example: $\sum \log i \in \Theta(\sqrt[3]{n} \cdot \log n)$

THEOREMS

The sum runs $i = 1$ to $\lfloor \sqrt[3]{n} \rfloor$, and by the previous slide we may drop the floor.

UPPER

every term is at most $\log \sqrt[3]{n}$

$$\sum_{i=1}^{\sqrt[3]{n}} \log i < \sum_{i=1}^{\sqrt[3]{n}} \log \sqrt[3]{n}$$

$$= \sqrt[3]{n} \cdot \log \sqrt[3]{n}$$

$$= (\sqrt[3]{n} / 3) \cdot \log n$$

LOWER

keep only the top half of the terms

$$\sum_{i=1}^{\sqrt[3]{n}} \log i > \sum_{i=\sqrt[3]{n}/2}^{\sqrt[3]{n}} \log (\sqrt[3]{n} / 2)$$

$$= (\sqrt[3]{n} / 2) \cdot \log (\sqrt[3]{n} / 2)$$

$$\approx (\sqrt[3]{n} / 6) \cdot \log n$$

$$(\sqrt[3]{n} / 6) \log n < \sum_{i=1}^{\lfloor \sqrt[3]{n} \rfloor} \log i < (\sqrt[3]{n} / 3) \log n \Rightarrow \Theta(\sqrt[3]{n} \log n)$$

Same two moves as every Θ proof today: bound every term above by the largest, and keep only the big half to bound below.

Extra Credit

BEFORE YOU GO

We showed earlier that $\log n! \leq n \log n$, **so** $\log n! \in O(n \log n)$.

Show $\log n! \in \Omega(n \log n)$

and conclude $\log n! \in \Theta(n \log n)$.

Hint: use the move from the last two slides. Keep only the largest half of the factors in $n! = (1)(2)\cdots(n)$ and bound each of them below.

Common Mistakes

BEFORE YOU GO

- 01 Treating big O as an estimate. It is an *upper bound*: binary search is $O(2^n)$, truthfully and uselessly. Use Θ when you mean the actual rate.
- 02 Proving a negative claim by trying witnesses. To show $f \notin O(g)$ you must rule out *every* pair — that is a contradiction proof.
- 03 Swapping C_1 and C_2 in a Θ proof. The definition is $C_2|g| \leq |f| \leq C_1|g|$ — lower constant second.
- 04 Forgetting that k exists. A bound that fails for small n is still a valid big-O bound.

Recap

LECTURE 07

01 $f \in O(g)$ means $|f| \leq C|g|$ past some k . Producing C and k is the proof.

02 A polynomial is Θ of its highest power. Everything below it is absorbed into a constant.

03 $1 \preccurlyeq \log n \preccurlyeq n \preccurlyeq n \log n \preccurlyeq n^2 \preccurlyeq 2^n \preccurlyeq n!$, and no link in the chain reverses.

04 Ω is the floor, Θ is both at once. $f \in \Theta(g)$ iff $f \in O(g)$ and $f \in \Omega(g)$.

05 Every Θ proof today used two moves: bound all terms by the largest, and keep only the big half.

Bubble sort is $\Theta(n^2)$. Binary search is $\Theta(\log n)$. Now you can say why.